

# TypeScript

---

JavaScript with Safety Rails

CSCI 39548 — Practical Web Development

## Section Goals

---

- ▶ Set up TypeScript locally and compile `.ts` to `.js`
- ▶ Understand why static types catch bugs earlier
- ▶ Annotate variables, functions, arrays, and objects
- ▶ Define interfaces, unions, and literal types
- ▶ Use modules, async types, and DOM type assertions

## Section Items

---

- ▶ Local Setup and Compile
- ▶ Why TS? The Bug JS Misses
- ▶ Primitive Types and Inference
- ▶ Typed Functions
- ▶ Arrays, Objects, and Interfaces
- ▶ Unions, Literals, and Narrowing
- ▶ Type Assertions — `as` and `!`
- ▶ Modules
- ▶ Async with Types

## Section Items

---

### ► Local Setup and Compile

- Why TS? The Bug JS Misses
- Primitive Types and Inference
- Typed Functions
- Arrays, Objects, and Interfaces
- Unions, Literals, and Narrowing
- Type Assertions — `as` and `!`
- Modules
- Async with Types

# What TypeScript Is

---

- ▶ JavaScript with a static type checker bolted on
- ▶ You write `.ts` files, the compiler produces `.js` files
- ▶ The browser and Node.js never see your types — they disappear at compile time
- ▶ TypeScript is a **build-time tool**, not a runtime language

# The Compile Flow

## 1. Write TypeScript

You write a `.ts` file

user.ts

```
type User = {  
  id: number;  
  name: string;  
  active: boolean;  
};  
  
function greet(user: User): string {  
  return `Hello, ${user.name}!`;  
}  
  
const u: User = {  
  id: 1,  
  name: "Ada",  
  active: true,  
};
```

.ts

## 2. tsc Compiler

TypeScript compiler  
checks types...



TypeScript  
Compiler

...

```
type User = { ... }  
user: User  
  : number  
  : string  
  : boolean  
  : ...  
  : ...
```

Types disappear here  
(compile time only)

## 3. JavaScript Output

...and emits plain JavaScript  
(no types)

user.js

```
function greet(user) {  
  return `Hello, ${user.name}!`;  
}  
  
const u = {  
  id: 1,  
  name: "Ada",  
  active: true,  
};
```

.js

## 4. Run Anywhere

JavaScript runs in Node.js  
or the browser



Node.js

or



Browser

# Installing TypeScript

---

```
npm install -g typescript  
  
tsc --version
```

- ▶ Install globally with npm
- ▶ Verify with `tsc --version`

## Your First .ts File

---

```
const name: string = "Sourya";  
const age: number = 21;  
console.log(`Hello, ${name}! You are ${age} years old.`);
```

- ▶ Same JavaScript you already know, plus : type annotations
- ▶ Open snapshot-01/hello.ts and walk through it



## Compile and Run

---

```
tsc hello.ts          # produces hello.js
node hello.js         # "Hello, Sourya! You are 21 years old."
```

- ▶ tsc reads .ts, emits .js
- ▶ Open both files side by side — the types are GONE in the .js

## When Types Catch Errors

---

```
tsc broken.ts  
# error TS2345: Argument of type 'string' is not  
# assignable to parameter of type 'number'.
```

- ▶ The compiler shows you the exact file, line, and column
- ▶ You fix the bug before the code ever runs

## Watch Mode

---

```
tsc --watch hello.ts
```

- ▶ Recompiles automatically on every save
- ▶ The productive workflow — leave it running in a terminal

## Key Takeaway

---

- ▶ TypeScript is a development tool — not a runtime language
- ▶ The browser and Node never see your types
- ▶ Types exist to help YOU and your team catch mistakes early
- ▶ They vanish completely in the compiled output

## Section Items

---

- Local Setup and Compile

### ► **Why TS? The Bug JS Misses**

- Primitive Types and Inference
- Typed Functions
- Arrays, Objects, and Interfaces
- Unions, Literals, and Narrowing
- Type Assertions — `as` and `!`
- Modules
- Async with Types

# Why TypeScript?

---

Bugs JS Can't Catch

# The Bug

---

```
function calculateTotal(bill, tip) {  
    return bill + tip;  
}  
  
const total = calculateTotal("100", "10");  
console.log(`Total: ${total}`);
```

Looks fine, right?

## The Result

---

- ▶ Expected: Total: \$110
- ▶ Actual: Total: \$10010
- ▶ "100" + "10" is string concatenation, not addition
- ▶ JavaScript runs it with no warning — the user sees the wrong total



## The Fix in JS

---

```
function calculateTotal(bill, tip) {  
    return Number(bill) + Number(tip);  
}
```

- ▶ You have to remember `Number()` every single time
- ▶ JS won't remind you — it just silently does the wrong thing

## The Same Code in TypeScript

---

```
function calculateTotal(bill: number, tip: number): number {  
    return bill + tip;  
}  
  
calculateTotal("100", "10");  
// Error: Argument of type 'string' is not assignable  
//      to parameter of type 'number'.
```

The compiler catches it before you ever run the code.

- ▶ Open `buggy.ts` — point at the red squiggle in VS Code
- ▶ Run `tsc buggy.ts` — see the precise error in the terminal

## The Value Proposition

---

- ▶ TS reads your code, figures out the types, and tells you where the bug is
- ▶ Bugs caught at edit time are  $100\times$  cheaper than bugs your users find
- ▶ You pay a small cost (annotations) for a huge benefit (compiler-verified correctness)

## Section Items

---

- Local Setup and Compile
- Why TS? The Bug JS Misses
- ▶ **Primitive Types and Inference**
  - Typed Functions
  - Arrays, Objects, and Interfaces
  - Unions, Literals, and Narrowing
  - Type Assertions — `as` and `!`
  - Modules
  - Async with Types

# Primitive Types

---

And Why You Barely Have to Type Them

# The Five Primitives

---

```
const name: string = "Alice";  
const age: number = 21;  
const active: boolean = true;  
const missing: null = null;  
const notSet: undefined = undefined;
```

string, number, boolean, null, undefined.

# Type Inference

---

```
const name = "Alice";      // TS knows: string
const age = 21;            // TS knows: number
const active = true;       // TS knows: boolean
```

- ▶ You almost never need to write annotations on variables
- ▶ TS infers the type from the value you assign
- ▶ This is the headline feature — the compiler does the work



## When to Annotate

---

| Situation                | Annotate?                        |
|--------------------------|----------------------------------|
| Function parameters      | Always                           |
| Declared without a value | Usually                          |
| Assigned immediately     | Rarely (inference handles it)    |
| Function return type     | Sometimes (to lock the contract) |

## null and undefined

---

- ▶ Both are real types in TypeScript
- ▶ `null` = intentionally empty
- ▶ `undefined` = not yet assigned
- ▶ They become powerful when combined with union types (coming soon)

## any — The Trap

---

```
let anything: any = "hello";  
anything = 42;  
anything.thisMethodDoesNotExist();  
// Compiles fine. Crashes at runtime!
```

- ▶ any turns off the type checker — anything goes
- ▶ TS stops helping — you're back to JavaScript's "hope for the best"

## unknown — The Safe Escape Hatch

---

```
let value: unknown = "hello";  
// value.toUpperCase(); // Error! Can't use directly.  
  
if (typeof value === "string") {  
    value.toUpperCase(); // OK -- narrowed to string  
}
```

- ▶ Like any, accepts anything
- ▶ Unlike any, forces you to check before use
- ▶ Reach for unknown, never any

- ▶ Run `tsc types.ts && node types.js`
- ▶ Watch the any crash at runtime
- ▶ Strongest argument against any

## Recap

---

- ▶ Annotate parameters — let TS infer everything else
- ▶ Reach for `unknown`, never `any`
- ▶ `any` compiles clean but crashes at runtime — the worst of both worlds

## Section Items

---

- Local Setup and Compile
- Why TS? The Bug JS Misses
- Primitive Types and Inference

### ► **Typed Functions**

- Arrays, Objects, and Interfaces
- Unions, Literals, and Narrowing
- Type Assertions — `as` and `!`
- Modules
- Async with Types

# Typed Functions

---

Parameters, Returns, and Function Types



## Annotate Params, Infer the Return

---

```
function add(a: number, b: number) {  
    return a + b; // TS infers return type: number  
}
```

- ▶ Always annotate parameters
- ▶ Let TS infer the return type most of the time

## Explicit Return Type

---

```
function add(a: number, b: number): number {  
    return a + b;  
}
```

- ▶ Add : `ReturnType` when you want to lock the contract
- ▶ Useful for public API functions, library code

## Optional Parameters

---

```
function greet(first: string, last?: string): string {  
    return last ? `${first} ${last}` : first;  
}  
  
greet("Alice");           // OK  
greet("Alice", "Smith");  // OK
```

- ▶ ? makes the parameter optional (type becomes `string | undefined`)
- ▶ Optional parameters must come AFTER required ones

## Default Parameters

---

```
function greet(name: string, greeting = "Hello") {  
    return `${greeting}, ${name}!`;  
}
```

- ▶ Type inferred from the default value
- ▶ No annotation needed — TS knows `greeting` is `string`

## Rest Parameters

---

```
function sum(...nums: number[]): number {  
    return nums.reduce((acc, n) => acc + n, 0);  
}  
  
sum(1, 2, 3, 4); // 10
```

Typed as an array: `...nums: number[]`.

## void

---

```
function logMessage(msg: string): void {  
    console.log(msg);  
}
```

- ▶ void = the function returns nothing useful
- ▶ Side-effect-only functions

## Function Type Aliases

---

```
type NumberTransformer = (n: number) => number;  
  
const double: NumberTransformer = (n) => n * 2;  
const square: NumberTransformer = (n) => n ** 2;
```

- ▶ Name a function signature for reuse
- ▶ This is what React's event handler types look like

- ▶ Walk through `functions.ts` — seven sections
- ▶ End with the typed `formatTime` at the bottom



## Section Items

---

- Local Setup and Compile
- Why TS? The Bug JS Misses
- Primitive Types and Inference
- Typed Functions
- ▶ **Arrays, Objects, and Interfaces**
  - Unions, Literals, and Narrowing
  - Type Assertions — `as` and `!`
  - Modules
  - Async with Types

# Shapes

---

Arrays, Objects, and Interfaces

## Array Types

---

```
const nums: number[] = [1, 2, 3];  
const names: Array<string> = ["Alice", "Bob"];
```

- ▶ Two equivalent syntaxes: `number[]` and `Array<number>`
- ▶ `number[]` is more common

# Tuples

---

```
const pair: [number, string] = [1, "Alice"];  
const [id, name] = pair; // destructuring works
```

- ▶ Fixed length, typed positions
- ▶ Use sparingly — objects with named keys are usually clearer

## Inline Object Types

---

```
function printUser(user: { name: string; age: number }) {  
    console.log(`${user.name} is ${user.age}`);  
}
```

- ▶ Define the shape right where you need it
- ▶ Fine for one-off use — gets unwieldy for reuse

## Optional Properties

---

```
function printUser(user: { name: string; phone?: string }) {  
    console.log(user.name);  
    if (user.phone) {  
        console.log(user.phone);  
    }  
}
```

- ▶ ? makes a property optional
- ▶ Must check before using — TS enforces this

## readonly

---

```
interface Config {  
    readonly apiUrl: string;  
    readonly maxRetries: number;  
}  
  
const config: Config = { apiUrl: "...", maxRetries: 3 };  
config.apiUrl = "new"; // Error! readonly
```

- ▶ Prevents mutation after construction
- ▶ Compile-time only — not enforced at runtime

## interface Todo — The Centerpiece

---

```
interface Todo {  
  id: number;  
  text: string;  
  done: boolean;  
  priority: number;  
  minutes: number;  
}
```

- ▶ Same shape as our JS Todo — now with compiler enforcement
- ▶ Every function that touches a Todo knows exactly what it looks like



# The Refactor Superpower

---

- ▶ Add a new field to Todo
- ▶ Every callsite that creates a Todo lights up red
- ▶ The compiler walks you through every place you need to update
- ▶ No grep, no “find all” — the type system does it for you

## Extending Interfaces

---

```
interface TodoWithDueDate extends Todo {  
    dueDate: string;  
}
```

- ▶ Inherits all fields from `Todo`, adds new ones
- ▶ Great for specialization without duplication

## interface vs type

---

| Feature       | interface | type             |
|---------------|-----------|------------------|
| Object shapes | Best fit  | Works            |
| Extending     | extends   | & (intersection) |
| Unions        | No        | Yes              |
| Primitives    | No        | Yes              |

Rule: interface for object shapes, type for everything else.

- ▶ Walk through `shapes.ts`
- ▶ End with the typed todos: `Todo[]` and `.filter().reduce()`
- ▶ Hover over the callback's `t` — it's inferred as `Todo`

## Section Items

---

- Local Setup and Compile
- Why TS? The Bug JS Misses
- Primitive Types and Inference
- Typed Functions
- Arrays, Objects, and Interfaces
- ▶ **Unions, Literals, and Narrowing**
  - Type Assertions — `as` and `!`
  - Modules
  - Async with Types

# Unions and Narrowing

---

Smart Unions and How TS Reads Your `ifs`

## Union Types

---

```
let value: string | number;  
value = "hello";    // OK  
value = 42;         // OK  
value = true;       // Error!
```

- ▶ `string | number` means “this OR that”
- ▶ Before narrowing, you can only use members that exist on BOTH sides

# Narrowing

---

```
function show(x: string | number) {  
    if (typeof x === "string") {  
        console.log(x.toUpperCase()); // TS knows: string  
    } else {  
        console.log(x.toFixed(2));    // TS knows: number  
    }  
}
```

- ▶ The compiler reads your if statements
- ▶ Inside each branch, it knows the exact type
- ▶ The smart compiler moment



## Narrowing Tools

---

| Tool                               | Narrows by          |
|------------------------------------|---------------------|
| <code>typeof x === "string"</code> | Primitive type      |
| <code>x !== null</code>            | Eliminates null     |
| <code>"key" in obj</code>          | Object has property |
| <code>x instanceof Date</code>     | Class instance      |

## String Literal Unions

---

```
type Tier = "high" | "med" | "low";

function setTier(tier: Tier) { /* ... */ }
setTier("high");           // OK
setTier("medium");         // Error! Typo caught.
```

- ▶ Lock the set of valid values
- ▶ Anything else is a compile-time error — typo prevention

## Numeric Literal Unions

---

```
type Priority = 1 | 2 | 3;

interface Todo {
  id: number;
  text: string;
  done: boolean;
  priority: Priority;
  minutes: number;
}
```

Our Todo's priority is now type-safe — only 1, 2, or 3.

## null in Unions

---

```
function findUser(id: number): string | null {  
    if (id === 1) return "Alice";  
    return null;  
}  
  
const user = findUser(99);  
// user.toUpperCase(); // Error! Might be null.  
if (user !== null) {  
    user.toUpperCase(); // OK -- narrowed to string  
}
```

Forces you to handle the missing case. No more “cannot read property of null” at runtime.

- ▶ Open `unions.ts` — hover variables inside vs outside `if` blocks
- ▶ Watch VS Code report `string` inside, `string | number` outside

## Recap

---

- ▶ Unions say “one of these”
- ▶ Narrowing proves which one
- ▶ Literal types lock the set of valid values

## Section Items

---

- Local Setup and Compile
- Why TS? The Bug JS Misses
- Primitive Types and Inference
- Typed Functions
- Arrays, Objects, and Interfaces
- Unions, Literals, and Narrowing
- ▶ **Type Assertions** — `as` and `!`
  - Modules
  - Async with Types

as **and** !

---

The Escape Hatches You'll See on Day One of React



## The Setup: `querySelector` Returns `null`

---

```
const el = document.querySelector("#name");  
// Type: Element | null
```

- ▶ `querySelector` might not find anything — returns `Element | null`
- ▶ TS forces you to handle the `null` before using the element

## Three Approaches

---

| Approach        | Syntax                              | When to use                |
|-----------------|-------------------------------------|----------------------------|
| Null check      | <code>if (el !== null)</code>       | Safest — always correct    |
| Non-null assert | <code>el!</code>                    | You control the HTML       |
| Type assert     | <code>el as HTMLInputElement</code> | Narrow to specific subtype |

## Null Check

---

```
const el = document.querySelector("#name");  
if (el !== null) {  
    el.textContent = "Hello";  // OK -- narrowed  
}
```

Safest approach — handles the null case explicitly.

## Non-Null Assertion !

---

```
const el = document.querySelector("#name")!;  
el.textContent = "Hello"; // OK -- you promised it exists
```

- ▶ ! tells TS: “I know this isn’t null, trust me”
- ▶ Use when you control the HTML and are certain the element exists

## Type Assertion `as`

---

```
const input = document.querySelector("#name")  
    as HTMLInputElement;  
input.value = "Alice"; // OK -- narrowed
```

- ▶ `as` casts from a general type to a specific one
- ▶ Needed because `querySelector` returns generic `Element`

## The as Pitfall

---

```
const x = "hello" as unknown as number;  
x.toFixed(2); // Compiles fine. Crashes at runtime!
```

- ▶ as is not a check — it's a cast
- ▶ TS is trusting you — don't lie
- ▶ If you're wrong, the crash happens at runtime with no warning

- ▶ Walk through `dom.ts` — all three approaches
- ▶ Run `index.html` in the browser — type a name, click Go, watch the heading update
- ▶ Bridge to React: `useRef<HTMLInputElement>(null) + ref.current?.focus()`  
— same idea, cleaner syntax

## Section Items

---

- Local Setup and Compile
- Why TS? The Bug JS Misses
- Primitive Types and Inference
- Typed Functions
- Arrays, Objects, and Interfaces
- Unions, Literals, and Narrowing
- Type Assertions — `as` and `!`

### ► **Modules**

- Async with Types



# Modules

---

How Files Talk to Each Other

# Why Modules Matter

---

```
import { useState } from "react";  
import { Todo } from "../types";  
import { formatTodo } from "../utils";
```

- ▶ Every React file starts with imports
- ▶ Modules are how you split code across files and share it

## Named Exports

---

```
// todo-utils.ts
export function formatTodo(todo: Todo): string { ... }
export function totalMinutes(todos: Todo[]): number { ... }
```

```
// main.ts
import { formatTodo, totalMinutes } from "./todo-utils.js";
```

- ▶ As many named exports per file as you want
- ▶ Importer matches the exact name

## Default Exports

---

```
// TodoList.ts  
export default class TodoList { ... }
```

```
// main.ts  
import TodoList from "./TodoList.js";
```

- ▶ At most one per file
- ▶ Importer picks any name they want

## Type Exports

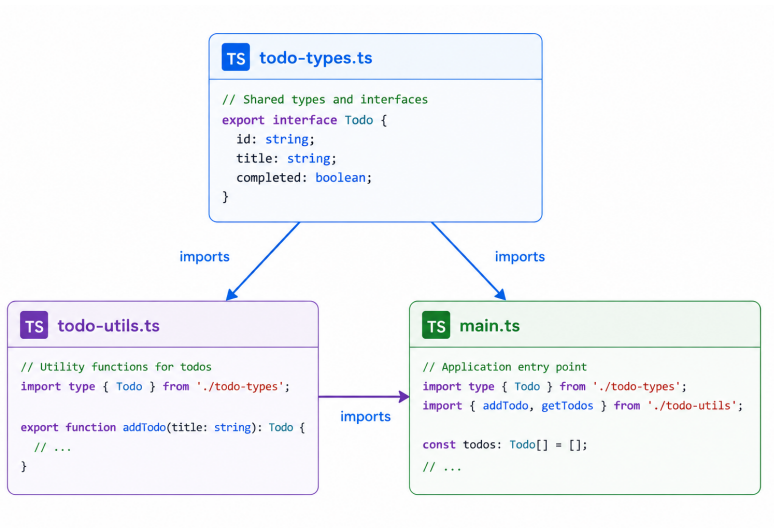
---

```
// todo-types.ts
export interface Todo {
  id: number;
  text: string;
  done: boolean;
  priority: 1 | 2 | 3;
  minutes: number;
}
```

- ▶ Interfaces and types export with the same syntax as values
- ▶ Foreshadow: shared type files in React projects

# The Import Graph

---



- ▶ Open all three files in VS Code (split panes)
- ▶ Walk through the imports
- ▶ Run `tsc main.ts && node main.js` — tsc follows the imports and compiles all three

## The .js Quirk

---

```
// WRONG -- TS won't find this at runtime
import { Todo } from "./todo-types.ts";

// RIGHT -- use .js extension
import { Todo } from "./todo-types.js";
```

- ▶ Import paths use .js, not .ts
- ▶ TS compiles to .js — the import paths must match the output
- ▶ Students WILL hit this — call it out now



## Named vs Default

---

- ▶ **Named** for most things — explicit, greppable, refactor-friendly
- ▶ **Default** for the file's “main thing” (a class, a component)
- ▶ The whole project's dependency graph is visible at the top of each file

## Section Items

---

- Local Setup and Compile
- Why TS? The Bug JS Misses
- Primitive Types and Inference
- Typed Functions
- Arrays, Objects, and Interfaces
- Unions, Literals, and Narrowing
- Type Assertions — `as` and `!`
- Modules

### ► Async with Types

# Async with Types

---

Typing Promises and Fetch

## Promise<T>

---

```
const p: Promise<string> = fetch(url)
    .then((res) => res.text());
```

- ▶ Read as “a future T”
- ▶ `Promise<string>` = “eventually a string”

## Async Function Return Types

---

```
async function loadName(): Promise<string> {  
    const response = await fetch(url);  
    const data = await response.json();  
    return data.name;  
}
```

- ▶ Async functions always return `Promise<T>`
- ▶ TS infers the `T` — explicit annotation is optional but clarifying

## Typing fetch

---

```
interface ApiTodo {  
  userId: number;  
  id: number;  
  title: string;  
  completed: boolean;  
}  
  
const response = await fetch(url);  
const todos = await response.json() as ApiTodo[];
```

- ▶ `response.json()` returns `Promise<any>` — untyped
- ▶ You declare the shape, cast with `as`
- ▶ This is the boundary where types meet untyped data

## catch (err: unknown)

---

```
try {  
    const data = await loadTodos();  
} catch (err: unknown) {  
    if (err instanceof Error) {  
        console.error(err.message);  
    }  
}
```

- ▶ `catch (err: unknown)` is the modern safe pattern
- ▶ Narrow with `instanceof Error` before using `.message`

- ▶ Run `tsc async.ts && node async.js`
- ▶ Watch the out-of-order output
- ▶ “Async means completion order, not source order”



## Bridge to TanStack Query

---

- ▶ Next time: a library that does all this typed-fetch dance for you
- ▶ `useQuery<Todo[]>({ queryFn: fetchTodos })` — one line
- ▶ You're learning the raw version first so the abstraction makes sense

## What You Can Now Do

---

- ▶ Set up TS locally and compile ✓
- ▶ Annotate variables and functions ✓
- ▶ Define interfaces ✓
- ▶ Use unions, literals, narrowing ✓
- ▶ Handle `Element | null` with `as / !` ✓
- ▶ Export and import across files ✓
- ▶ Type async functions and `fetch` calls ✓

## Same Engine, Evolving Syntax

---

- ▶ The JS Todo and the TS Todo are the same shape
- ▶ Same logic, fewer surprises
- ▶ The compiler now guards every boundary

## What's Next

---

- ▶ **React** — component-based UI with JSX
- ▶ **Vite** — fast build tool (replaces manual tsc)
- ▶ **TanStack Query** — typed data fetching, simplified
- ▶ **Tailwind CSS** — utility-first styling

# Assignment

---

- ▶ Assignment details to be announced